EXP 36: Write a Prolog Program to implement Best First Search algorithm.

AIM

To write and implement a Prolog program to perform Best First Search using heuristic values to find the path from the initial state to the goal state.

Program:

%%writefile bestfirst.pl

```
% Graph connections
edge(a, b).
edge(a, c).
edge(b, d).
edge(b, e).
edge(c, f).
edge(c, g).
edge(d, h).
edge(e, h).
edge(f, h).
edge(g, h).

% Heuristic values
heuristic(a, 7).
heuristic(b, 6).
heuristic(c, 4).
heuristic(d, 5).
heuristic(e, 3).
heuristic(f, 2).
heuristic(g, 3).
heuristic(h, 0).

% Best First Search
bestfirst(Start, Goal, Path) :-
```

```
    search([[Start]], Goal, RevPath),
    reverse(RevPath, Path).

search([[Goal|Rest]|_], Goal, [Goal|Rest]).

search([ [Node|Rest] | Others], Goal, Path) :-
    findall([Next,Node|Rest],
        (edge(Node, Next),
         \+ member(Next, [Node|Rest])),
        Children),
    append(Others, Children, NewQueue),
    sort_queue(NewQueue, SortedQueue),
    search(SortedQueue, Goal, Path).

% Sort paths by heuristic value
sort_queue(Queue, Sorted) :-
    predsort(compare_paths, Queue, Sorted).

compare_paths(Order, [A|_], [B|_]) :-
    heuristic(A, HA),
    heuristic(B, HB),
    compare(Order, HA, HB).
!swipl -q -s bestfirst.pl -g "bestfirst(a, h, Path), write('Best First Search Path: '), writeln(Path), halt."
```

Out[put:



Exp 37: Write the prolog program for Medical Diagnosis.

AIM

To write and implement a Prolog program for Medical Diagnosis using symptoms and logical rules to identify possible diseases.

Program:

```
prolog_code = """
% Symptoms
symptom(fever).
symptom(cough).
symptom(headache).
symptom(sneezing).
symptom(body_pain).

% Medical diagnosis rules
disease(flu) :-
   symptom(fever),
   symptom(cough),
   symptom(body_pain).

disease(common_cold) :-
   symptom(sneezing),
   symptom(cough).

disease(malaria) :-
   symptom(fever),
   symptom(headache),
   symptom(body_pain).

% Display diagnosis
diagnosis :-
   disease(Disease),
```

```
    write('Possible Disease: '),
    write(Disease), nl,
    fail.

diagnosis.
"""

with open("medical.pl", "w") as f:
    f.write(prolog_code)

!swipl -s medical.pl -g "diagnosis, halt."
```

Output:

```
Possible Disease: flu
Possible Disease: common_cold
Possible Disease: malaria
```

Exp 38 Write a Prolog Program for forward Chaining. Incorporate required queries.

AIM

To write and implement a Prolog program to demonstrate Forward Chaining using facts and rules to derive new conclusions from the available knowledge base by incorporating appropriate queries.

Program:

```
prolog_code = """
% Declare dynamic predicate
:- dynamic fact/1.

% Initial facts
fact(sunrise).
fact(morning).

% Rules
```

```
rule(day) :-
   fact(sunrise),
   fact(morning).

rule(bright) :-
   fact(day).

rule(active) :-
   fact(day),
   fact(bright).

% Forward Chaining
forward :-
   rule(X),
   \\+ fact(X),
   assertz(fact(X)),
   write('New Fact: '), write(X), nl,
   !,
   forward.

forward :-
   write('Forward Chaining Completed'), nl.

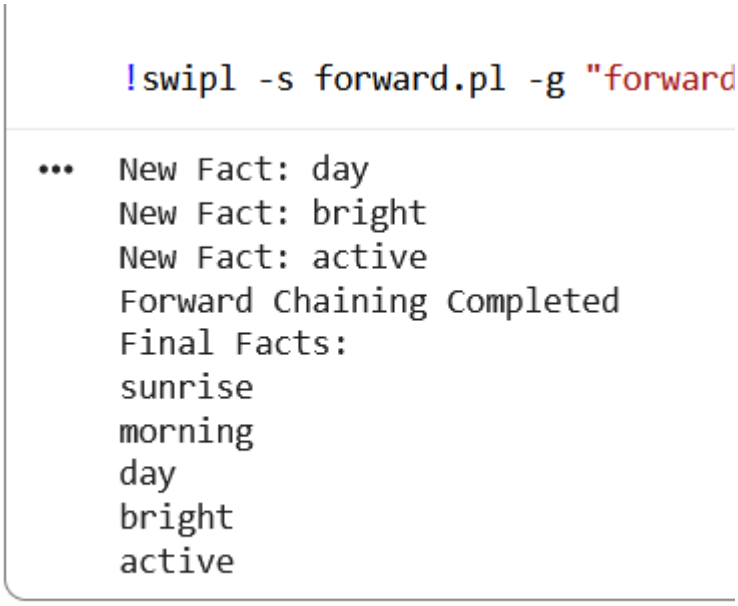
% Display all facts
show_facts :-
   fact(X),
   write(X), nl,
   fail.

show_facts.
"""
```

```
with open("forward.pl", "w") as f:
    f.write(prolog_code)
```

```
!swipl -s forward.pl -g "forward, write('Final Facts:'), nl, show_facts, halt."
```

Output:



Exp 39 Write a Prolog Program for backward Chaining. Incorporate required queries.

AIM

To write and implement a Prolog program to demonstrate Backward Chaining using facts and rules to prove a given goal by working backward from the conclusion to the required conditions.

Program:

```
prolog_code = """
% Initial facts
fact(fever).
fact(cough).
fact(body_pain).

% Rules
disease(flu) :-
    fact(fever),
```

```
    fact(cough),
    fact(body_pain).

disease(common_cold) :-
    fact(cough),
    fact(sneezing).

% Backward Chaining Query
diagnosis(Goal) :-
    disease(Goal).

% Display diagnosis
check(Goal) :-
    (diagnosis(Goal) ->
        write(Goal), write(' is proved.'), nl
    ;
        write(Goal), write(' cannot be proved.'), nl
    ).
"""

with open("backward.pl", "w") as f:
    f.write(prolog_code)

!swipl -s backward.pl -g "check(flu), check(common_cold), halt."
```

Output:

```
with open('backward.pl', 'w') as f:
    f.write(prolog_code)

!swipl -s backward.pl -g "check(flu), check(common_cold), halt."
```

```
flu is proved.
common_cold cannot be proved.
```

Exp 40 Create a Web Blog using Word press to demonstrate Anchor Tag, Title Tag, etc.

AIM

To create a web blog using WordPress and demonstrate basic HTML tags such as Anchor Tag, Title Tag, Heading Tag, Paragraph Tag and Image Tag.

Program:

```
from IPython.display import display, HTML


html_code = """
<!DOCTYPE html>
<html>
<head>
  <title>My Technology Blog</title>
</head>
<body>

  <h1>Welcome to My Technology Blog</h1>

  <h2>Introduction</h2>

  <p>Technology makes our life easier.
  It helps us communicate and learn new things.</p>

  <h2>Popular Technologies</h2>

  <p>Artificial Intelligence</p>
  <p>Machine Learning</p>
  <p>Web Development</p>

  <h2>Useful Website</h2>

  <a href="https://www.google.com" target="_blank">
```

```
    Visit Google
    </a>

    <br><br>

    <img src="https://placehold.co/300x150"
    alt="Technology Image">

</body>
</html>
"""

display(HTML(html_code))
```

Output:

```
display(HTML(html_code))
```

Welcome to My Technology Blog

Introduction

Technology makes our life easier. It helps us communicate and learn new things.

Popular Technologies

Artificial Intelligence
Machine Learning
Web Development

Useful Website

Visit Google

